# Contents

**Endpoint Inventory** **1**
Conventions . . . 1
1. Identity — `/api/v1/auth` . . . 2
2. Profile and social graph — `/api/v1/me`, `/api/v1/users` . . . 3
3. Catalog — `/api/v1/tracks` . . . 4
4. Track Vote — `/api/v1/events` . . . 4
5. Playlist Editor — `/api/v1/playlists` . . . 5
6. Control Delegation — `/api/v1/devices` . . . 6
7. Playback — `/api/v1/playback-sessions` . . . 7
8. Audit and operations . . . 7
9. Realtime — client-to-server hub methods . . . 8
10. What is deliberately absent . . . 9
11. Open items . . . 9
Related . . . 9

# Endpoint Inventory

The API contract, written before the code. This is [[delivery-plan]] Phase 3, and it is what unblocks the UI developer — the two workstreams meet here and nowhere else.

**Status: design, nothing implemented.** Once code exists, `docs/api/openapi.json` generated from that code becomes the machine-readable contract and this file becomes the human overview. They must not diverge; a change lands in both in the same pull request.

Rules this document obeys: [[rest-api-design]] · [[openapi-documentation]] · [[signalr-realtime]] · [[authentication-authorization]] · [[owasp-api-top10-checklist]]

---

## Conventions

| Aspect | Value |
|---|---|
| Base path | `/api/v1` |
| Format | JSON, `camelCase`, UTF-8, UTC ISO-8601 instants |
| Errors | RFC 9457 `application/problem+json`, one shape, always with `traceId` |
| Collections | `{ "items": [...], "page": 1, "pageSize": 20, "total": 137 }`, `pageSize` capped at 100 |
| Ids in JSON | strings, always |
| Auth | `Authorization: Bearer <access token>` |
| Mutation headers | `X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version` — **required**, missing means `400` |
| Optimistic concurrency | `ETag` on the resource, `If-Match` on the mutation, mismatch means `412` |

### Authorisation notation

Two gates, both evaluated server-side on every call ([[sdd]] §13.2):

| Symbol | Gate 1 — identity | Gate 2 — visibility and licence |
|---|---|---|
| `anon` | none | none |
| `auth` | any authenticated user | none |
| `self` | the caller owns the resource | none |
| `owner` | the caller owns the aggregate | none |
| `member` | the caller is an attendee or collaborator | visibility resolved first |
| `licence` | the caller passes the aggregate's `LicenceMode` check | includes presence proof where required |

**A resource the caller may not see returns `404`, never `403`.** `403` would confirm it exists.

**Status codes used**

`200` ok · `201` created (+ `Location`) · `202` accepted (generic, non-enumerating) · `204` no content · `400` malformed or missing client headers · `401` unauthenticated · `403` authenticated but not permitted · `404` absent **or hidden from this caller** · `409` conflict · `412` precondition failed (stale `If-Match`) · `422` semantically invalid · `429` rate limited

---

## 1. Identity — `/api/v1/auth`

Subject V.1, V.5. Rules: [[authentication-authorization]], [[social-oauth]].

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/auth/registrations` | `email`, `password`, `displayName` | `202` | `400`, `422`, `429` | `anon` |
| POST | `/auth/email-verifications` | `token` | `204` | `400`, `410` expired, `429` | `anon` |
| POST | `/auth/verification-emails` | `email` | `202` | `429` | `anon` |
| POST | `/auth/sessions` | `email`, `password` | `200` token pair | `401` generic, `429` | `anon` |
| POST | `/auth/sessions/social` | `provider`, `idToken` | `200` token pair | `401`, `422`, `429` | `anon` |
| POST | `/auth/tokens` | `refreshToken` | `200` rotated pair | `401`, `429` | `anon` |
| DELETE | `/auth/sessions/current` | | `204` | `401` | `auth` |
| GET | `/auth/sessions` | — | `200` list | `401` | `self` |
| DELETE | `/auth/sessions/{sessionId}` | | `204` | `401`, `404` | `self` |
| POST | `/auth/password-reset-requests` | `email` | `202` | `429` | `anon` |
| POST | `/auth/password-resets` | `token`, `newPassword` | `204` | `400`, `410`, `422`, `429` | `anon` |
| GET | `/auth/social-links` | | `200` list | `401` | `self` |
| POST | `/auth/social-links` | `provider`, `idToken` | `201` | `409` already linked, `422` | `auth` |
| DELETE | `/auth/social-links/{provider}` | | `204` | `409` last sign-in method | `self` |

**Design notes**

- **`202` on registration, verification resend and password reset is deliberate.** A `409 email already exists` is an account-enumeration oracle. The response is identical whether or not the address is known; the mailbox tells the real story. Cost: a typo'd address gives no immediate feedback. Accepted, and stated at defence.
- `/auth/tokens` performs refresh **rotation**. Presenting an already-rotated refresh token revokes the whole family and returns `401` — this is the session-theft detector (T-03).
- `/auth/password-resets` revokes every refresh token for that account on success.
- `/auth/sessions/social` both registers and signs in. Identity comes from the validated **ID token**, never an access token ([[ADR-0008-oidc-social-login]]).
- `DELETE /auth/social-links/{provider}` refuses with `409` when it would leave the account with no usable sign-in method.
- Every row here is rate limited per account **and** per IP.

---

## 2. Profile and social graph — `/api/v1/me`, `/api/v1/users`

Subject V.1. The friends graph exists because the `Friends` visibility tier requires it.

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| GET | `/me` | — | `200` full own profile | `401` | `self` |
| PATCH | `/me/profile` | any subset of profile fields, each with its `visibility` | `200` | `422` | `self` |
| PUT | `/me/music-preferences` | `genres[]`, `artists[]` | `200` | `422` | `self` |
| GET | `/users` | query: `q`, `page`, `pageSize` | `200` paged, public fields only | `401` | `auth` |
| GET | `/users/{userId}` | — | `200` **filtered per field** | `404` | `auth` |
| GET | `/me/friends` | paged | `200` | `401` | `self` |
| DELETE | `/me/friends/{userId}` | — | `204` | `404` | `self` |
| GET | `/me/friend-requests` | query: `direction=incoming\|outgoing` | `200` paged | `401` | `self` |
| POST | `/me/friend-requests` | `targetUserId` | `201` | `409` duplicate or already friends, `404` | `auth` |
| PATCH | `/me/friend-requests/{requestId}` | `status`: `accepted \| declined` | `200` | `404`, `409` | `self` (recipient) |
| DELETE | `/me/friend-requests/{requestId}` | — | `204` | `404` | `self` (sender) |

**Design notes**

- `GET /users/{userId}` projects **per field** according to the requester's relationship (self / friend / stranger) against each field's `ProfileFieldVisibility`. The full row never leaves the server for the client to filter — OWASP API3.
- `PATCH /me/profile` sets both value and visibility in one call, so a new field cannot exist without a tier. A missing tier is `422`, never a silent default to public.

- `GET /users` search returns public fields only, paginated, rate limited. Unbounded user enumeration is OWASP API4.

---

## 3. Catalog — `/api/v1/tracks`

Subject III. Rules: [[youtube-integration]].

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| GET | `/tracks` | query: `q`, `page`, `pageSize` | 200 paged `TrackRef` | 429, 503 quota | `auth` |
| GET | `/tracks/{youtubeVideoId}` | | 200 `TrackRef` | 404, 503 | `auth` |

**Design notes**

- `GET /tracks` is the **only** path that can reach `search.list`, the expensive YouTube call. It is debounced client-side, cached server-side, and rate limited. `GET /tracks/{id}` prefers the cache, then `videos.list`.
- The YouTube API key is server-side only. The client never calls YouTube for metadata.
- Quota exhaustion surfaces as `503` with a clear problem type, not a `500`.
- A `TrackRef` is `{ youtubeVideoId, title, channel, durationSeconds, thumbnailUrl }`. No media.

---

## 4. Track Vote — `/api/v1/events`

Subject V.2.1. The contended write path. Rules: [[realtime-architecture]], [[ADR-0014-location-time-licence]].

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/events` | `name`, `visibility`, `licence`, `startsAtUtc`, `endsAtUtc` | 201 | 422 | `auth` |
| GET | `/events` | query: `scope=public\|mine\|invited`, paging | 200 paged | 401 | `auth` |
| GET | `/events/{eventId}` | | 200 | 404 if hidden | `member` |
| PATCH | `/events/{eventId}` | metadata subset | 200 | 404, 412 | `owner` |
| DELETE | `/events/{eventId}` | | 204 soft delete | 404 | `owner` |
| PUT | `/events/{eventId}/visibility` | `visibility`, `mode`, `presenceProof`, `geofence`, `window` | 200 | 422 | `owner` |
| POST | `/events/{eventId}/invite-codes` | `ttlSeconds` | 201 short-lived code | 404, 429 | `owner` |
| GET | `/events/{eventId}/invitations` | paging | 200 | 404 | `owner` |
| POST | `/events/{eventId}/invitations` | `userId` | 201 | 404, 409 | `owner` |
| DELETE | `/events/{eventId}/invitations/{userId}` | | 204 | 404 | `owner` |

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/events/{eventId}/attendances` | | `201` | `404`, `409` | `auth` if public, `member` if private |
| DELETE | `/events/{eventId}/attendances/current` | | `204` | `404` | `self` |
| GET | `/events/{eventId}/queue` | paging, vote-ordered | `200` | `404` | `member` |
| POST | `/events/{eventId}/tracks` | `youtubeVideoId` | `201` suggestion | `404`, `409` duplicate, `422` | `licence` |
| DELETE | `/events/{eventId}/tracks/{suggestionId}` | | `204` | `404` | `owner` or suggester |
| PUT | `/events/{eventId}/tracks/{suggestionId}/vote` | optional `presence` | `201`/`200` | `404`, `409`, `422`, `429` | `licence` |
| DELETE | `/events/{eventId}/tracks/{suggestionId}/vote` | | `204` | `404` | `self` |
| PATCH | `/events/{eventId}/now-playing` | `suggestionId` | `200` | `404`, `409` | `owner` or delegate |

**Design notes**

- **Voting is `PUT`, not `POST`.** A vote is an idempotent fact: "this user votes for this track". A double submit from a flaky mobile network is then harmless — `201` the first time, `200` after. `POST` would create a second row and make the race worse.
- The vote's uniqueness is enforced by `ux_votes_event_id_track_id_user_id`, and the duplicate is detected **by the database**, never by a read-then-write check ([[ef-core-persistence]]).
- The vote count is recomputed from the source rows inside the transaction. Nothing increments a counter held in memory.
- `presence` carries `{ coordinates?, eventCode? }` and is required only when the licence is `LocationAndTimeRestricted`. The server checks it against the event's geofence and window, using **server time**. The coordinate is used and discarded — it is never written to the vote row (threat T-21) and never logged.
- `POST /events/{eventId}/event-codes` is owner-only and rate limited; codes are short-lived and single-use per user.
- The broadcast on `EventHub` happens **after** commit, never before.
- **Attendance is not a gate on a public event.** Subject V.2.1 says that when an event is public, *all* users can find it and vote. `POST /attendances` records presence for the roster; it is never required before `PUT .../vote` on a public event. On a private event, the invitation is the gate, not the attendance.

---

## 5. Playlist Editor — `/api/v1/playlists`

Subject V.2.3. The second contended write path.

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/playlists` | `name`, `visibility`, `licence` | `201` | `422` | `auth` |
| GET | `/playlists` | query: `scope=public\|mine\|shared`, paging | `200` paged | `401` | `auth` |
| GET | `/playlists/{playlistId}` | | `200` + `ETag` | `404` if hidden | `member` |
| PATCH | `/playlists/{playlistId}` | `If-Match`, subset | `200` | `404`, `412` | `licence` |

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| DELETE | `/playlists/{playlistId}` | — | `204` soft delete | `404` | `owner` |
| PUT | `/playlists/{playlistId}/licence` | `visibility`, `mode` | `200` | `422` | `owner` |
| GET | `/playlists/{playlistId}/collaborators` | — | `200` | `404` | `member` |
| POST | `/playlists/{playlistId}/collaborators` | `userId`, `canEdit` | `201` | `404`, `409` | `owner` |
| DELETE | `/playlists/{playlistId}/collaborators/{userId}` | — | `204` | `404` | `owner` |
| GET | `/playlists/{playlistId}/tracks` | — | `200` + `ETag` | `404` | `member` |
| POST | `/playlists/{playlistId}/tracks` | `trackId`, `afterTrackId?` | `201` | `404`, `422` | `licence` |
| PATCH | `/playlists/{playlistId}/tracks/{playlistTrackId}` | `afterTrackId` or `beforeTrackId` | `200` | `404`, `409`, `412` | `licence` |
| DELETE | `/playlists/{playlistId}/tracks/{playlistTrackId}` | — | `204` | `404` | `licence` |

**Design notes**

- **Reorder is a `PATCH` with `If-Match`.** The client sends the `ETag` it last read; a stale one gets `412` and the client reloads. Without this, two simultaneous moves silently overwrite each other — the exact failure subject V.2.3 grades.
- Position is a sparse ordering key computed **between the neighbours named in the request**, so inserting between two tracks does not rewrite the list.
- `afterTrackId: null` means "move to the head". Sending both `afterTrackId` and `beforeTrackId` is `422`.
- Conflicts that survive a reload-and-reapply return `409` rather than guessing the user's intent.
- `PlaylistHub` broadcasts `TrackMoved` with the new version after commit.

---

## 6. Control Delegation — `/api/v1/devices`

Subject V.2.2. Licence is **per device**, bound to the account.

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/devices` | `platform`, `model`, `appVersion` | `201` | `409` already registered, `422` | `auth` |
| GET | `/devices` | — | `200` own devices | `401` | `self` |
| DELETE | `/devices/{deviceId}` | — | `204` | `404` | `self` |
| GET | `/devices/{deviceId}/control` | — | `200` + `ETag` current controller | `404` | `self` or controller |
| PUT | `/devices/{deviceId}/control` | `granteeUserId` | `200` | `404`, `409`, `412` | `owner` |
| DELETE | `/devices/{deviceId}/control` | — | `204` released | `404` | `owner` or controller |
| GET | `/me/control-grants` | — | `200` grants held by me | `401` | `self` |

**Design notes**

- Control is a **single row with a state machine and a concurrency token** ([[sdd]] §11.2). Two simultaneous claims: one wins, the other gets `412`. The loser must lose **visibly** — a silent overwrite is the bug.
- `granteeUserId` must be a friend of the owner. Otherwise `422`.
- `DELETE .../control` is allowed to both the owner (revoke) and the current controller (release).
- `POST /devices` derives its values from the client context headers, which are mandatory anyway.

---

## 7. Playback — `/api/v1/playback-sessions`

Subject V.2.1 (a highly-voted track plays earlier) and V.2.2 (control is per device). Model and rationale: [[ADR-0017-playback-session]].

| Method | Path | Body | Success | Failures | Auth |
|---|---|---|---|---|---|
| POST | `/playback-sessions` | `deviceId`, `sourceType`, `sourceId` | `201` + `ETag` | `404`, `409` device already has a session, `422` | `owner` of the device |
| GET | `/playback-sessions/{sessionId}` | — | `200` + `ETag` | `404` | `member` of the source |
| GET | `/devices/{deviceId}/playback-session` | — | `200` + `ETag` | `404` | `self` or controller |
| POST | `/playback-sessions/{sessionId}/advance` | `expectedNowPlayingId` | `200` + `ETag` | `404`, `409` queue empty, `412` stale | controller |
| PATCH | `/playback-sessions/{sessionId}` | `nowPlayingId` (explicit jump) | `200` | `404`, `412` | controller |
| DELETE | `/playback-sessions/{sessionId}` | — | `204` stop playback | `404` | `owner` of the device |

**Design notes**

- **`expectedNowPlayingId` makes advance idempotent.** A duplicate call from a flaky mobile network finds its expectation stale and gets the current state back — it never skips a second track. Without it, a retry silently loses a song.
- The server picks the next item: for an `Event`, the highest vote count with ties broken by earliest suggestion; for a `Playlist`, the next ordering key. **The client reports that the track ended; it never chooses what plays next.**
- `controllerUserId` on the session is what `PUT /devices/{id}/control` changes. That is the concrete meaning of "delegating music control".
- `NowPlayingChanged` broadcasts on `EventHub` or `PlaylistHub` **after** commit.
- A device holds at most one active session.

## 8. Audit and operations

| Method | Path | Success | Auth | Note |
|---|---|---|---|---|
| GET | `/me/activity` | `200` paged own audit records | `self` | optional; the **write** side is the graded requirement (V.6), the read side is convenience |

| Method | Path | Success | Auth | Note |
|---|---|---|---|---|
| GET | `/health` | `200` | `anon` | liveness, minimal body |
| GET | `/health/ready` | `200` / `503` | `anon` | readiness: database and provider reachability |
| GET | `/openapi/v1.json` | `200` | `anon` | the generated contract; interactive UI only in `Development` |

Audit records are **written** by a pipeline behaviour on every command — that is subject V.6, and it is not optional. See [[logging-observability]].

---

## 9. Realtime — client-to-server hub methods

Server-to-client events are in [[sdd]] §9.2 and [[signalr-realtime]]. Hub methods are API endpoints with a different transport, and the same authorisation applies.

| Hub | Method | Effect | Auth |
|---|---|---|---|
| `EventHub` | `JoinEvent(eventId)` | join group `event:{eventId}` | visibility + licence, **re-checked on reconnect** |
| `EventHub` | `LeaveEvent(eventId)` | leave the group | `auth` |
| `PlaylistHub` | `JoinPlaylist(playlistId)` | join group `playlist:{playlistId}` | visibility + licence, re-checked on reconnect |
| `PlaylistHub` | `LeavePlaylist(playlistId)` | leave the group | `auth` |
| `ControlHub` | — | auto-joins `user:{userId}` on connect | `auth` |

**Design notes**

- **Mutations do not happen over the hub.** Every state change goes through the REST endpoints above, so one authorisation pipeline and one validation pipeline cover both transports. The hub carries joins, leaves and server-pushed facts.
- A client asking to join `playlist:42` proves nothing. Group membership is authorised server-side, every time.
- Authentication uses the same JWT, via the documented SignalR `access_token` query parameter on the WebSocket handshake.
- **Client context is captured at the handshake**, not per invocation — a hub call carries no HTTP headers. A handshake without `platform`, `device` and `appVersion` is rejected, mirroring the `400` a REST mutation gets for the same omission.
- **Join and leave are audited** by a hub filter writing the same record as the REST behaviour ([[ADR-0019-hub-action-auditing]]). Connect and disconnect are not — mobile reconnect noise would bury deliberate actions.

---

## 10. What is deliberately absent

| Not present | Why |
|---|---|
| Any endpoint returning another user's private data | subject V.6; there is no "admin view" in this project |
| Bulk or unpaginated list endpoints | OWASP API4; every list is capped |
| A `PUT /users/{id}` style full replace | invites mass assignment; profile updates are `PATCH` with explicit fields |
| Media streaming or proxying | playback is client-side YouTube; we store only `TrackRef` |
| Direct database or Supabase access from a client | subject V.3 makes the backend the sole guardian of truth |
| Mutation over SignalR | keeps one authorisation and validation path |

---

## 11. Open items

| # | Item | Resolved by |
|---|---|---|
| E1 | Concrete request and response schemas per endpoint | the OpenAPI document, generated from code |
| E2 | Exact `PlaybackSession` and control state names | the S7 slice ([[sdd]] §11.2) |
| E3 | Whether `GET /me/activity` ships | drop it first if S9 is tight; the write side is what is graded |
| E4 | Pagination cursor vs offset for the event queue | offset assumed; revisit if the queue grows large |

---

## Related

[[sdd]] §9 · [[delivery-plan]] · [[rest-api-design]] · [[signalr-realtime]] · [[owasp-api-top10-checklist]] · [[musicroom-backend]] · [[musicroom-mobile]]